

From Vibe Coding to Spec-Driven Agentic Engineering

A Hands-On Implementation using Claude Code

PART III

Instructor: Muhammad Arif Butt, PhD

About The Instructor



Dr. Muhammad Arif Butt
Asst. Prof. Department of Data Science
University of Punjab, Lahore
arif@pucit.edu.pk
https://arif.excaliat@gmail.com
https://arifbutt.me
https://www.linkedin.com/in/dr-arif-butt/
https://youtube.com/learnwitharif
https://github.com/arifpucit
https://huggingface.co/arif-butt

- **Education:**
  - Graduation from Pakistan Military Academy, Kakul
  - MPhil CS from University of Punjab, Lahore
  - PhD CS from University of Punjab, Lahore
- **Experience:**
  - Served in field/staff/instructional posts in Pakistan Army
  - Assistant Professor, Department of Data Science
  - Founder Excaliat (https://excaliat.com/en)
  - Founder FalconHunt (https://falconhunt.org/)
  - Co-Founder Tbox Solutionz (https://tboxsolutionz.com/)
- **Research Interest:**
  - Embedded and Real-Time Operating Systems
  - Vulnerability Analysis, Binary Exploitation & Exploit Development
  - AI-Driven Cybersecurity and Securing AI systems

Scheme of Sessions

Part- I: Overview of Agentic Coding Assistants

Part– II: Claude Ecosystem, Installing & using Claude Code

Part– III: Slash Commands, Context Window & Claude Directory

Part– IV: A Vulnerable Web Application & its Attack Vectors

Part– V: Workflow of Implementing Features & Fixing Bugs

Part– VI: Skill Market place and Writing your Own Skills

Part– VII: Creating and using Subagents and Agent Teams

Part– VIII: Model Context Protocol

Part– IX: Creating and using Hooks

Part– X: Creating and using Plugins



Built-in Slash Commands

Instructor: Muhammad Arif Butt, PhD

From Manual Prompts to Slash Commands

Slash Commands are shortcuts you type inside Claude session, starting with /, that triggers a specific predefined action or workflow, without writing a full prompt

THE PROBLEM · Manual Workflows

You retype the same long, detailed prompt for every task.

"Explain this function..."
break down the inputs, outputs and logic, with clear examples

"Write unit tests..."
use pytest, cover edge cases, and report coverage

"Refactor this code..."
improve readability, remove redundancy, optimise

... then type it all again for the next task.

THE COST

- Slow
- Error-prone
- Repetitive
- Inconsistent

Slow, redundant, and easy to get wrong.

THE SHORTCUT

/tests

expands into

FULL PREDEFINED PROMPT

Write thorough unit tests with pytest, cover edge cases, and report coverage.

Claude runs it

Workflow runs

One short word stands in for a whole prompt — and the work runs automatically.

THE SOLUTION · Slash Commands

One command triggers the entire predefined workflow.

/explain	**Structured explanation** inputs, outputs & logic
/tests	**Full test suite** edge cases + coverage
/refactor	**Cleaner code** readable & efficient
/debug	**Bug found & fixed** trace, patch, verify

THE PAYOFF

- Fast
- Consistent
- Efficient
- Productive

Faster and repeatable — saves time, boosts productivity.

Four Kinds of / in Claude Code

Built-in slash commands
Fixed actions baked into the CLI

Control the session itself — switch models, manage context, sign in. The CLI runs them directly; the model isn't involved.

WHERE

No files — compiled into Claude Code

`/help` `/model` `/clear` `/config` `/context` `/compact` `/login`

You run them — Claude can't trigger them. 60+ ship with Claude Code; list them with `/help`.

Custom slash commands
A reusable prompt you write

A Markdown file whose contents become a prompt template, invoked by typing `/name`. Supports `$ARGUMENTS`. The older, simpler format.

WHERE

`.claude/commands/<name>.md` (project)
`~/.claude/commands/<name>.md` (personal)

`/deploy` `/fix-issue` `/optimize`

Manual only — runs just when you type it. Legacy format, still supported.

Built-in skills
Skills that ship with Claude

Each is an instruction set Claude runs with its tools. Invoke with `/name` — or Claude fires it itself when your request matches.

WHERE

Bundled with Claude Code · marked Skill in /commands

`/code-review` `/debug` `/commit` `/pdf`

The short description stays in context so Claude knows it exists; the full body loads only on use. ~10 ship with Claude Code.

Custom skills
A SKILL.md folder you write

YAML frontmatter + steps, plus optional scripts and files. Invoke with `/name`, or let Claude load it automatically when work matches.

WHERE

`.claude/skills/<name>/SKILL.md` (project)
`~/.claude/skills/<name>/SKILL.md` (personal)

`/summarize-changes` `/migrate-component`

Recommended format (v2.1.101+). Legacy `/commands` still work; if names clash, the skill wins.

- **Core Difference:** A custom slash command is invoke-only: nothing happens until you type /name. A skill adds an always-loaded description, so Claude can also trigger it on its own when the task matches — and it can bundle scripts and supporting files.
- **Why Skills are now Preferred:** Skills are a superset of commands — manual /name plus autonomous use — with progressive disclosure: only the short description sits in context, the body loads on demand. One unified system replaces two.

Built-in Slash Commands

Command	Description
/help	Show usage help and the full list of available commands.
/login	Sign in to your Anthropic account.
/logout	Sign out from your Anthropic account.
/theme	Change the color theme — light, dark, or accessible variants.
/model	Switch the AI model and save it as the default for new sessions.
/effort	Set the reasoning effort level, from low up to max.
/status	View version, model, account, and connectivity at a glance.
/doctor	Diagnose and verify your installation and settings.
/config	Open Settings to adjust theme, model, output style, and preferences.
/permissions	Manage allow, ask, and deny rules for tool permissions.

Built-in Slash Commands (cont...)

Command	Description
/rename	Rename the current session and show the name on the prompt bar.
/clear	Start a new conversation with empty context.
/btw	Ask a quick side question without adding to the conversation.
/resume	Reopen an earlier conversation by ID or name.
/rewind	Roll the code and/or conversation back to a previous checkpoint.
/recap	Generate a one-line summary of the current session.
/usage	Show session cost, plan usage limits, and activity stats.
/context	Visualize what is currently filling the context window.
/compact	Summarize the conversation so far to free up context.
/export	Export the current conversation as plain text.

Hands on Practice: Slash Commands



Claude Code's Permission Modes

1 Default — Approval mode

`the blank state · no indicator shown`

Claude pauses for your approval before any consequential action — editing a file, running a command. You **approve or reject every prompt**. Best for watching over Claude's shoulder.

● Asks before every action

2 Accept Edits

`⏵⏵ accept edits on`

Claude **applies file edits and basic filesystem changes automatically**, without asking. Other Bash / shell commands still stop and prompt you. Speeds up the editing flow.

● Auto-applies edits

3 Plan mode

`⏸ plan mode on`

Claude is restricted to **read-only**. It researches and proposes a plan but makes no changes until you approve. Approving exits plan mode and begins editing.

● Read-only · no changes

4 Auto mode

`auto mode on`

The most autonomous mode — Claude **executes without permission prompts**. Useful for large refactors or long repetitive tasks. Requires Claude Code v2.1.83+.

● No prompts · full autonomy

Two things to know about cycling modes

- Press **Shift+Tab** to cycle through these modes in order; the active one shows as an indicator at the bottom of your terminal. Type `/plan` to jump straight into Plan mode mid-conversation.
- More autonomy means **more speed but less oversight**. Accept Edits still prompts for non-edit Bash commands, and the full `bypassPermissions` ("YOLO") mode is intentionally kept out of the cycle by default.

Scopes of Permission Rules

1 Enterprise / Managed

`/etc/claude-code/managed-settings.json`

Applies to: Every project, every user on the machine

Set by an administrator at the machine level. Highest precedence — cannot be overridden by any user or project.

Outside the repository

2 User / Global

`~/.claude/settings.json`

Applies to: All of your projects on this machine

Personal to your account, stored in your home directory. Lowest precedence — applies when nothing more specific does.

Outside the repository

3 Project — Shared

`.claude/settings.json`

Applies to: This project · everyone on the team

Team baseline that lives inside the project. Checked into source control and shared with all collaborators.

Committed & pushed

4 Project — Local

`.claude/settings.local.json`

Applies to: This project · only you

Your personal overrides for this project. Claude Code automatically adds it to the project's .gitignore.

Git-ignored (stays local)

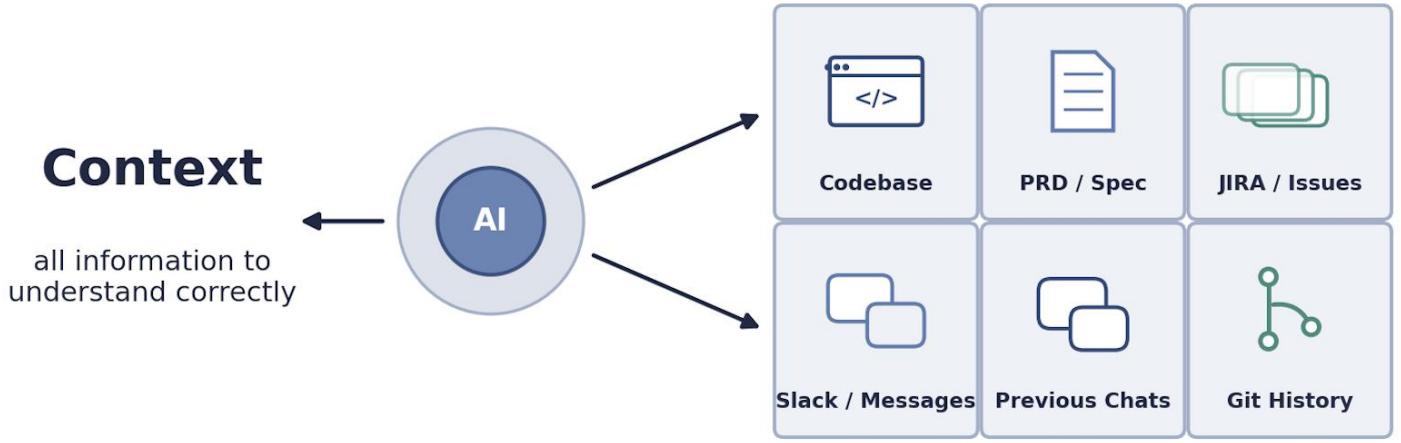
Two principles that govern all scopes

- Rules merge across scopes — they are not replaced. A deny rule always overrides an allow rule, no matter which file it sits in.
- Only the shared project file is meant for GitHub. Confirm .claude/settings.local.json appears in .gitignore so personal rules are not leaked.

Hands on Practice: /permissions



Context Window Management



Instructor: Muhammad Arif Butt, PhD

Context Window: The Mental Model

It's a document passed back and forth

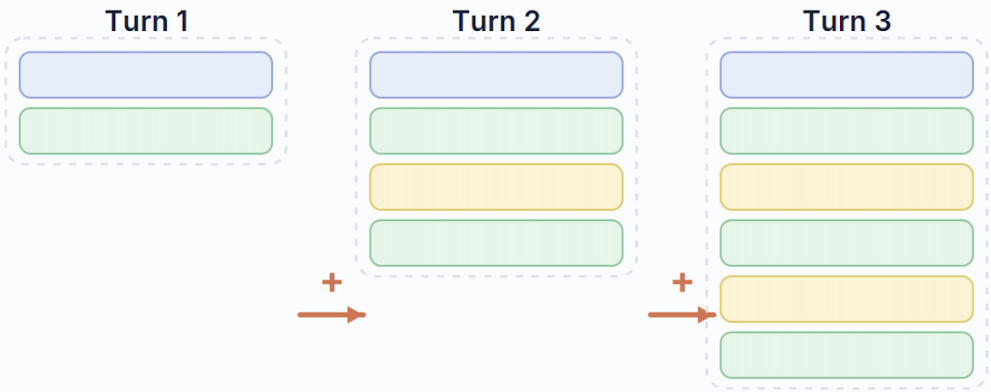
You aren't really chatting — each turn re-sends the entire history as one input.

- **Every turn, your whole history**
  — prompts, replies, files read, tool outputs — is bundled and sent as one big input.
- **The model reads all of it**
  generates a reply, and that reply is appended to the bundle.
- **Then the whole thing is sent again**
  on the next turn. Nothing is dropped on its own.
- **A 10-minute irrelevant detour**
  still sits in the bundle and is re-sent every single turn.

WHY COST COMPOUNDS

Message #50 re-sends messages #1–49.
The longer the session, the more every new turn costs.

THE SAME BUNDLE, GROWING EACH TURN

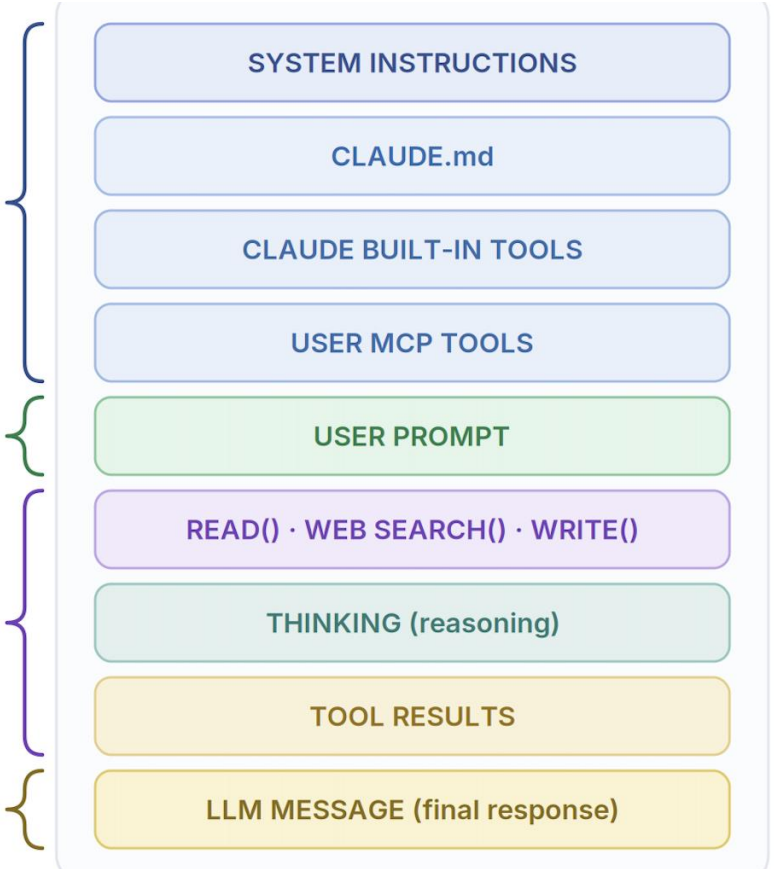


Same context window — it only gets bigger, never smaller on its own.

Context Window: Anatomy

You see two things. Many more are loaded.

Behind your prompt and the reply sits a stack of hidden context — all of it counted.



What you actually see

Only the user prompt and the final response.

What's hidden but still counts

- Anthropic's system instructions + your CLAUDE.md
- Built-in tool and MCP tool definitions
- Every file read, search, and command output
- The model's own step-by-step reasoning

The key idea

Everything in the stack — instructions, definitions, file reads, search results, the reasoning — lives in the window and eats into the same token budget.

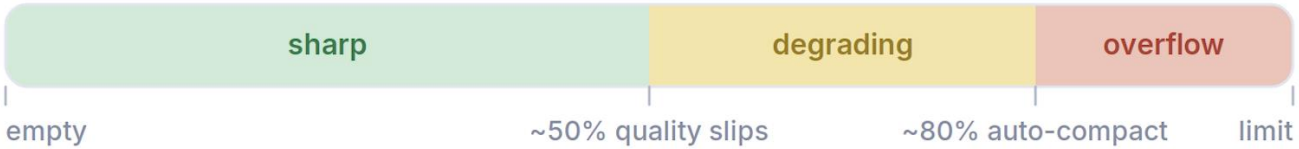
Try it: `/context` shows what is filling the window right now.

Context Window: Growth & Degradation

More context isn't more capability

As the window fills, the model has more to track — and quality quietly slips.

HOW PERFORMANCE TRACKS WITH FILL LEVEL



- **Tool results are the sneakiest bloat**
  every file read and command output is dumped into the window in full.
- **Quality slips around half-full**
  this is "context rot" — it starts well before the hard limit.
- **The model loses the thread**
  earlier constraints and decisions get crowded out by sheer volume.

THE ROUGH RELATIONSHIP

$$\text{performance} \approx \frac{\text{correctness}^2 \times \text{completeness}}{\text{size of context}}$$

Grow the denominator and the whole score falls.

Counting in tokens

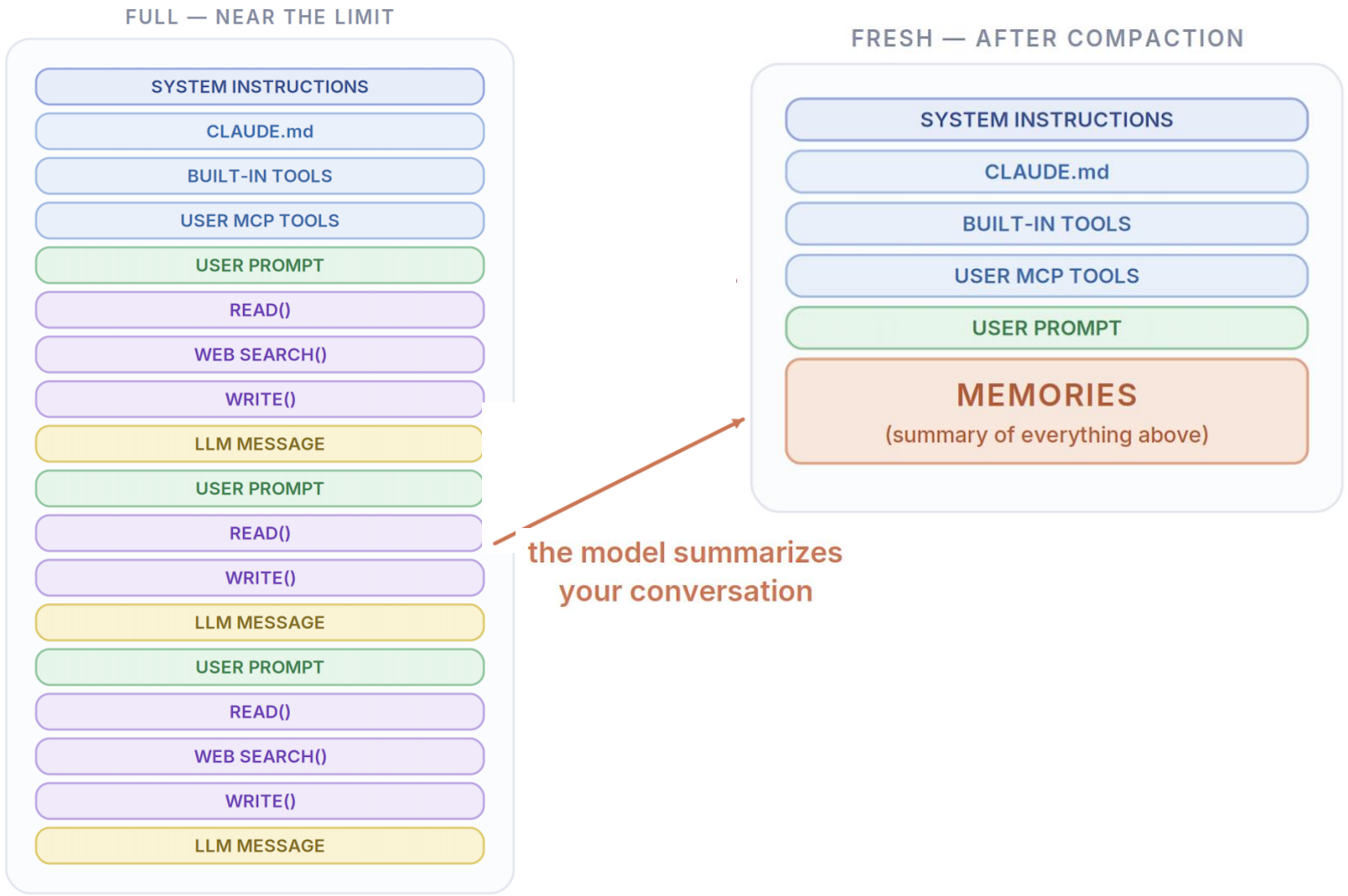
1 token ≈ ¾ of a word.

Limits depend on model + plan: commonly 200,000, up to 1,000,000 (1M) on extended-context models.

Context Window: Auto-Compaction

Auto-compaction: the automatic reset

Approaching the limit, Claude Code summarizes the history and continues from the summary.



It works — but rarely ideal

A decision from three steps ago, or an early constraint, can quietly get dropped.

It often fires mid-task in an already-degraded state, so the summary isn't the model's best work.

Markus's analogy: “like training while you sleep” — imperfect, but the intuition fits.

GOOD TO KNOW

Auto-compaction needs code execution enabled, and reserves a buffer near the top of the window.

Context Window: Take Control

Compact on purpose, not by accident

Manual compaction runs while there's still headroom — so the summary is far better.

`/compact`

Summarize now, keep the thread. Best at a phase boundary, with budget to spare.

`/compact remember the nonlinear-programming work`

Steer the summary toward what you actually care about.

`/clear`

Hard reset. Wipes history but keeps your code and CLAUDE.md. Use when switching tasks.

`/rewind (or Esc Esc)`

Jump back to a good state and drop everything after — cleaner than piling on "undo that" messages.

`/context`

See what is filling the window right now.

`/usage`

Check how many tokens you've spent.

QUICK DECISION GUIDE

Keep the thread → `/compact`

New, unrelated task → `/clear`

Went down a bad branch `/rewind`

Rhythm: compact at each milestone (every ~30–45 min). Aim for ~60% full, rather than waiting for auto-compaction near ~80%.

Context Window: Persistence

Let the disk hold your memory

Write progress to files, then start fresh sessions that read them back. Files persist; chat doesn't.



WHY THIS WINS

Each new session opens with a clean, full token budget — but every important decision is loaded back from disk.

Context Window: Best Practices

1 Keep your sessions short (one per feature)

/exit
End the current session

→

/clear
Clear the current context

→

/resume
Resume a previous session

Esc + Esc

Press Esc twice
Rewind your context to a previous appropriate point

2 Claude Code itself is an agent

Claude Code (Main Agent)

You can create sub agents for tasks that can be done in parallel

Sub Agent 1
Task: Research
Isolated Context Window

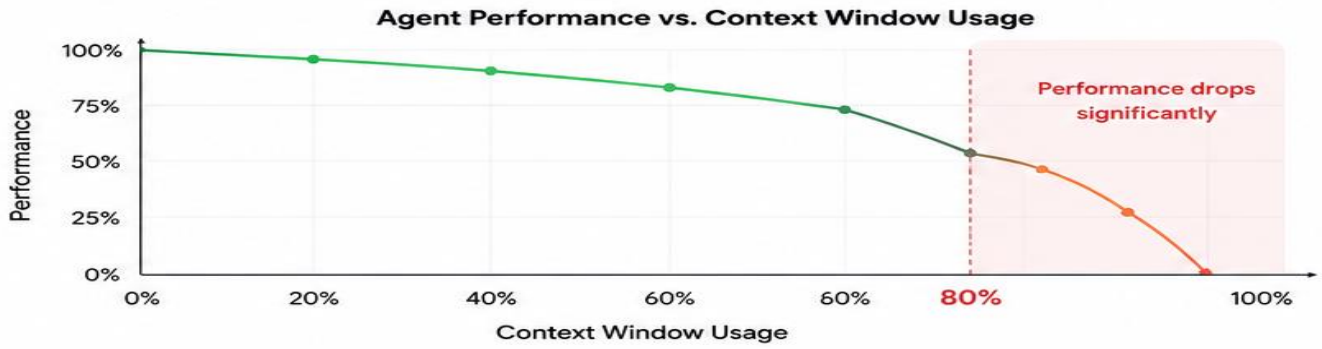
Sub Agent 2
Task: Analysis
Isolated Context Window

Sub Agent 3
Task: Implementation
Isolated Context Window

Each sub agent has its own isolated context window, completely separate from the main session.

3 Research shows: As the context window reaches up to 80% of its usage, the performance of the agent decreases.





Context Window: Cheat-Sheet

Habits & command cheat-sheet

Three habits keep the window healthy; six commands let you steer it.

1 **Long sessions degrade**
After ~20 turns of heavy output, start fresh. Turn 30 is noticeably worse than turn 3.

2 **Write state to files**
Important decisions go to a README or plan file — not only the chat. Files persist; context doesn't.

3 **Break work into chunks**
Run focused 5–10 turn sessions with one objective each, then start clean for the next.

Command reference

`/context`	see how full the window is
`/compact [notes]`	summarize, keep the thread
`/clear`	wipe history, keep your code
`/rewind`	jump back to a good state
`/usage`	check token spend
`CLAUDE.md`	persistent brief, auto-loaded

A NOTE ON NEWER MODELS

Recent Claude Opus models (4.6 and later) handle much of this automatically and degrade more slowly — but on long, complex work the habits still pay off

How To Give Context To Claude?

Visit this link to Explore the Growing of Context Window:

https://code.claude.com/docs/en/context-window

1 Type it in directly
Explicit, one-off context you push into a single prompt

@-mentions
`@auth.ts` `@src/`

Type @ then a file or folder name (tab-completion + fuzzy match) to pull its full contents into the prompt. Reference several at once.

Drag, drop & paste
`Shift+drag`

Drop a file into the prompt (Shift+drag in the VS Code panel) or paste an image from the clipboard — ideal for screenshots, mockups, PDFs & docs.

/add & paste code
`/add` `copy–paste`

Run /add <path> to attach a file by hand, or simply copy–paste code and terminal output straight into the prompt box.

2 Automatic — Claude sees it for you
Implicit context picked up without you attaching anything

IDE active file & selection
`Alt/Option+K`

In the VS Code / JetBrains extension Claude automatically sees your open file. Press Alt/Option+K to turn a selection into an @-mention with exact line numbers.

Auto memory
`MEMORY.md` `/memory`

Claude writes its own notes (MEMORY.md) from your corrections & preferences and reloads them every session. Inspect or edit with /memory.

3 Persistent — set once, loads every session
Context that survives across sessions and is shared with your team

CLAUDE.md
`/init` `./CLAUDE.md`

A markdown file at your project root with standards, build commands & rules. Loaded in full at the start of every session. Run /init to scaffold one; scopes run from user to project to organization.

MCP resources
`@server:…`

Connect external services — GitHub, databases, API docs — then reference their resources with @ so Claude reads live data beyond your local files.

4 Manage the context window
Keep the working memory clean so answers stay sharp

Context commands
`/context` `/compact` `/clear`

/context visualizes how the window is being used • /compact summarizes a long conversation to reclaim space • /clear starts fresh.

Instructor: Muhammad Arif Butt, PhD

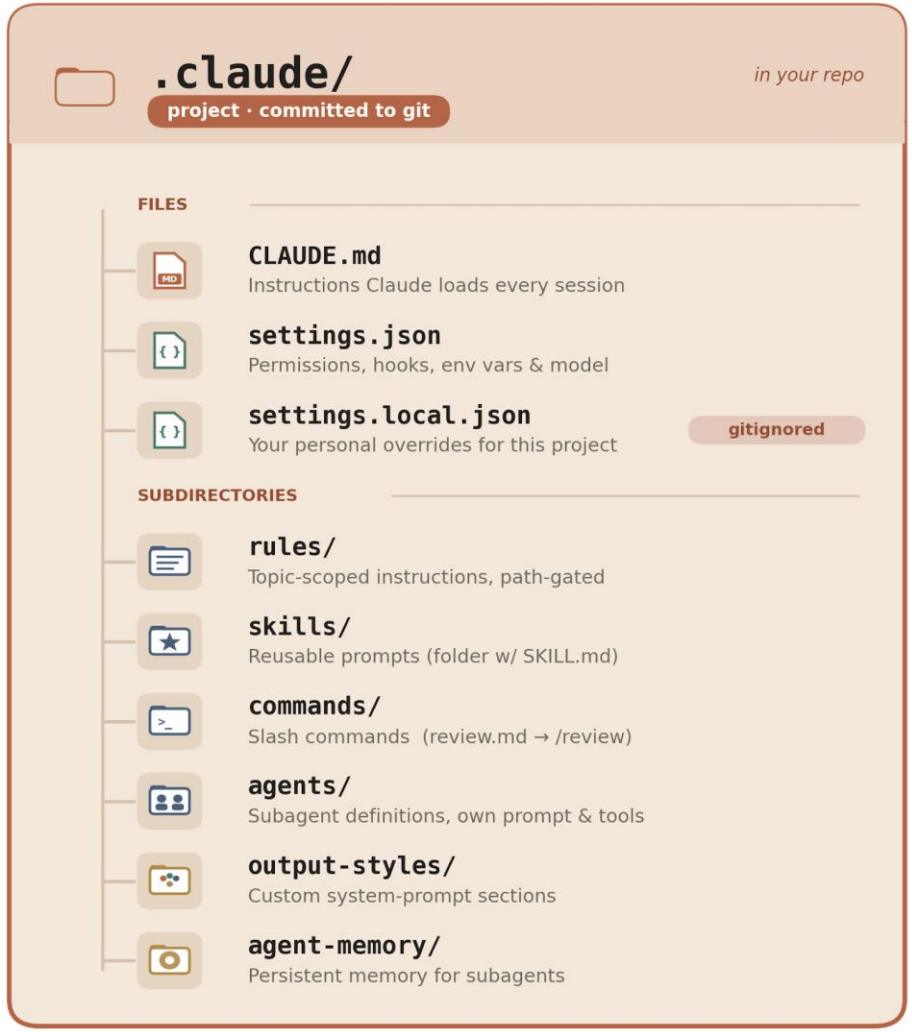
Hands on Practice: Context Window



The `.claude/` Directory Structure

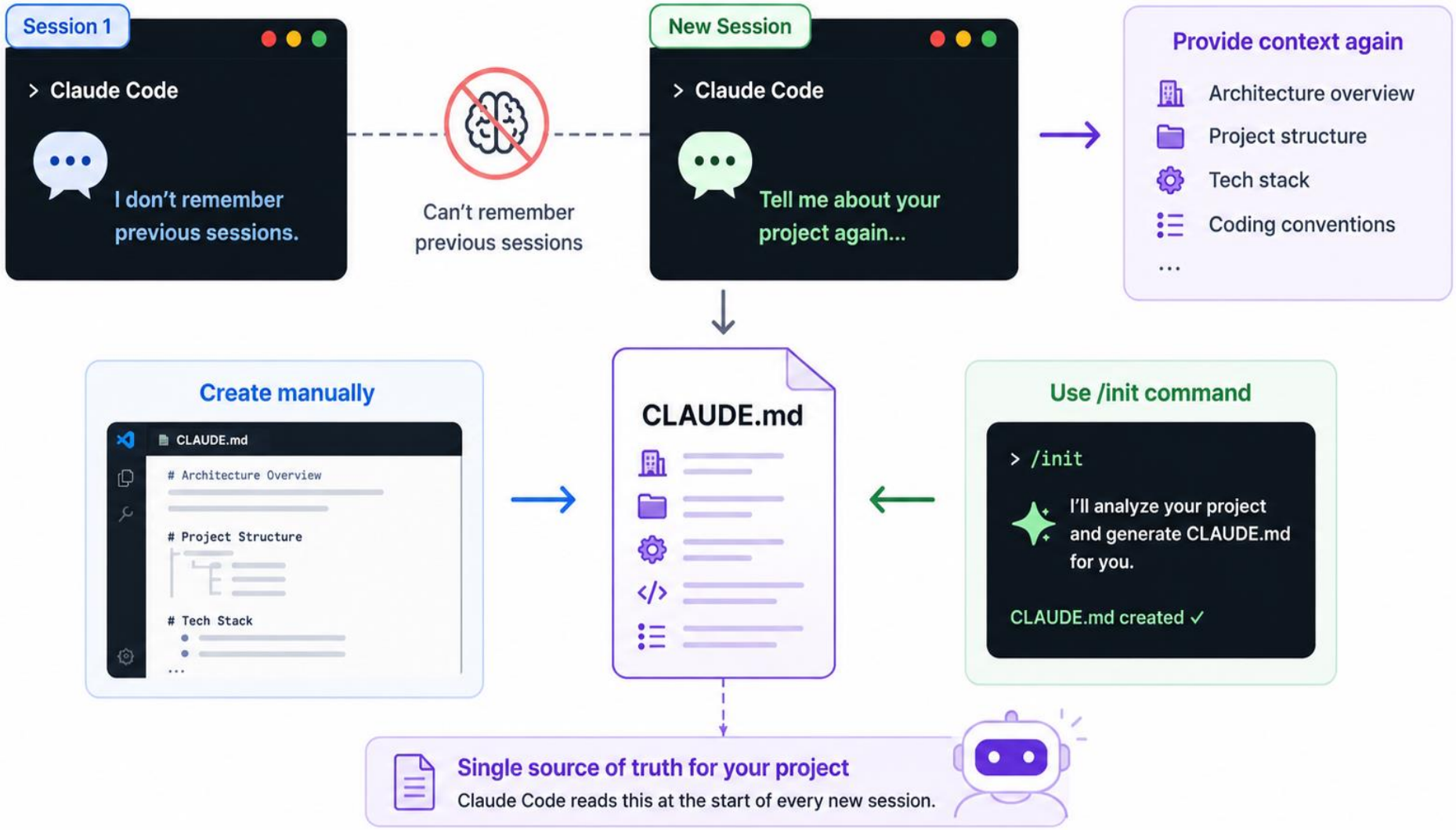
Explore the `.claude/` Directory

The [.claude directory] is the central control centre and memory store for Claude Code, and it stores project-specific guidelines, custom tools, permission settings, and session history





The Need of CLAUDE.md File

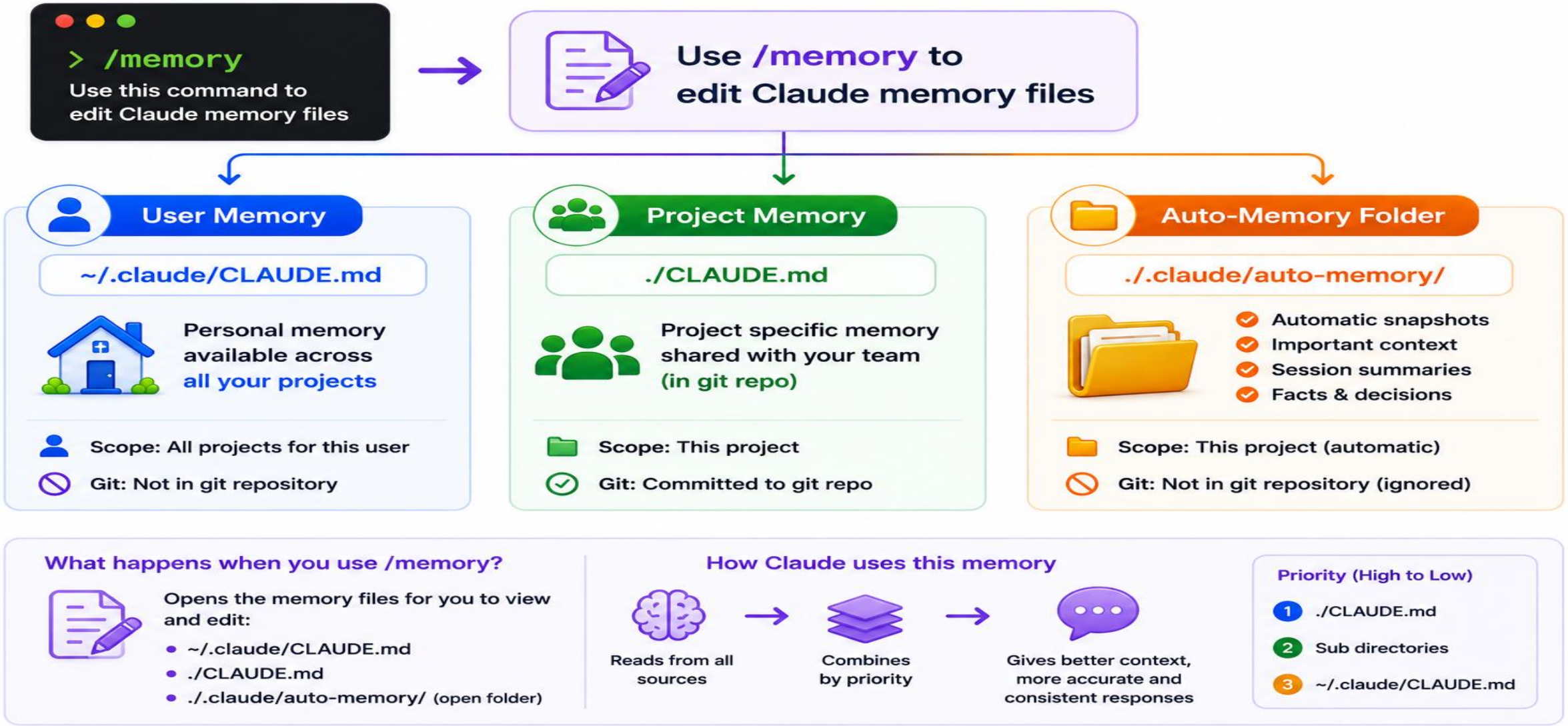


Multiple CLAUDE.md Files

Location	Path / Pattern	When Loaded	Shared With	Goes to Git Repo?
Project Root	`./CLAUDE.md` or `./.claude/CLAUDE.md`	Loaded at start of session	Team	Yes
Home Directory	`~/.claude/CLAUDE.md`	Loaded at start of session	All projects of this user	No
Sub Directories	(e.g., `./src/CLAUDE.md`)	Loaded on demand when Claude reads files in that directory	Team (via project)	Yes
.claude/rule/*.md	`.claude/rule/*.md`	Loaded on demand	Team (via project)	Yes
Local Overrides	`./CLAUDE.local.md`	Loaded at start of session	Only you (local)	No (git ignored)

- CLAUDE.md — Project root file
- ~/.claude/CLAUDE.md — User home config
- Sub directory CLAUDE.md — Context on demand
- .claude/rule/*.md — Custom rules (on demand)
- CLAUDE.local.md — Local overrides (git ignored)

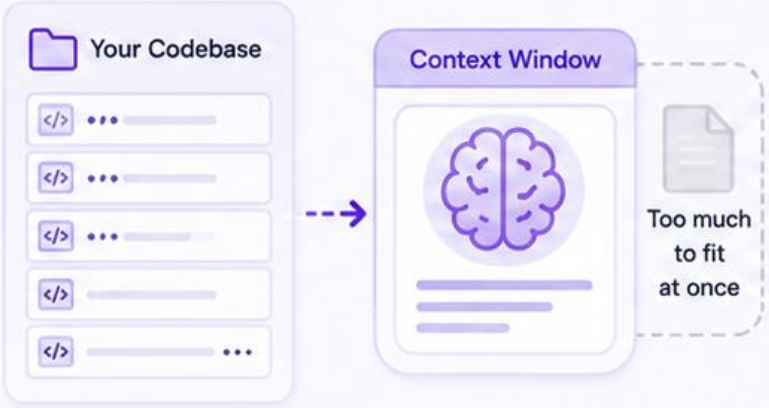
The /memory Command



Using Claude Code Effectively

1 The Context Window

Think of this as Claude's working memory.
It can hold a lot, but not everything at once.



This is where the "agentic" aspect comes in — Claude finds strategic ways to locate answers within your codebase without loading the entire thing into context.

2 It Asks for Permission

By default, Claude Code will ask you before running commands or making changes.
You're always in control.



Hands-on

Review and approve each step

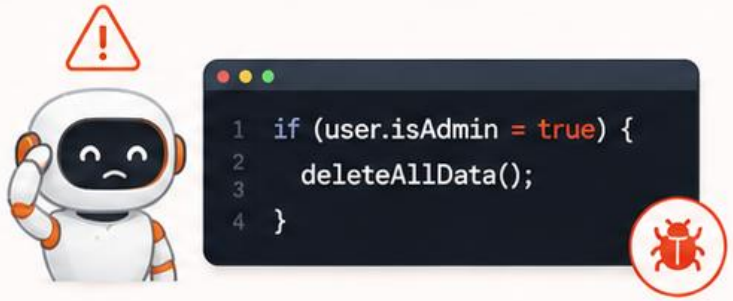
Hands-off

Trust Claude to proceed (with safeguards)

You're always in control.

3 It Can Make Mistakes

Just like any tool, Claude Code isn't perfect.
It might misunderstand your intent, introduce a bug, or over-engineer a solution.



Stay in the loop:

- Review changes
- Test your code
- Ask questions
- Catch issues early



The Sweet Spot Leverage Claude Code's power, keep oversight, and collaborate to build better software — faster and safer.

Hands on Practice



Q/A



Thank you!